

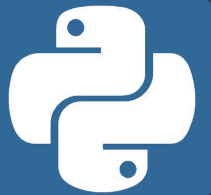
CURSO DE EXTENSÃO · PROGRAMAÇÃO PARA INICIANTES

Aula 01

Introdução à Programação e ao Pensamento Computacional

Antes de escrever a primeira linha de código, vamos entender como um computador pensa — e por que a lógica vem antes da linguagem.

Foco do curso: lógica de programação com Python



O que vamos ver hoje

Um roteiro simples: do “o que é um computador” até “por que aprender Python”.



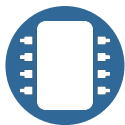
Programação e pensamento computacional

O que significa programar e como organizar o raciocínio antes do código.



Hardware x Software

As duas metades de todo sistema computacional.



Como o computador processa informação

Entrada → Processamento → Saída, e o papel da memória.



Binário e hexadecimal

Como tudo, no fundo, vira 0 e 1.



Dos circuitos ao código

Por que existem linguagens como Python — e por que isso ajuda.



Fechamento e próxima aula

Resumo do essencial e o que vem a seguir: algoritmos.

O que é programação?

Programar é **descrever, passo a passo, como um computador deve executar uma tarefa.**

“O computador é extremamente rápido, mas essencialmente literal: ele faz exatamente o que foi escrito — nem mais, nem menos.”

Por isso, programar não é só aprender uma linguagem. Antes da linguagem, existe a capacidade de:

- Entender o problema
- Definir uma sequência de passos
- Dividir em partes menores
- Estabelecer condições e repetições



Você pensa:

“Quero calcular a média de 3 notas.”

O computador precisa de:

1. Receber nota 1, 2 e 3
2. Somar as três notas
3. Dividir o total por 3
4. Exibir o resultado

Pensamento computacional

Uma forma estruturada de analisar problemas antes de escrever qualquer código.



Decomposição

Dividir um problema grande em partes menores e mais simples de resolver.



Abstração

Focar no que é relevante agora e ignorar detalhes desnecessários.



Reconhecimento de padrões

Perceber comportamentos ou estruturas que se repetem.



Algoritmo

Organizar uma sequência ordenada de passos para chegar à solução.

Antes de programar, é preciso saber exatamente o que queremos que o programa faça.

Hardware x Software

Todo sistema computacional é a soma dessas duas metades.



Hardware

A parte física do sistema.

- CPU e memória RAM
- SSD / HDD e placa-mãe
- GPU
- Teclado, mouse e monitor
- Dispositivos de rede

“Aquilo que se chuta quando dá problema” (é brincadeira tá? mas irão lembrar).



Software

Programas, instruções e dados que orientam o hardware.

Firmware

BIOS/UEFI — inicializa o hardware

Sistema Operacional

Windows, Linux, macOS, Android

Utilitários

Compactadores, anti-vírus, gerenciador de tarefas

Aplicativos

Navegador, editores de texto, jogos (programas de multimídia em geral)

“Aquilo que se xinga quando trava.”

Como o computador processa informação

O modelo de Von Neumann, simplificado: tudo o que um sistema faz cabe em três etapas.



E a memória? Dados e instruções ficam guardados numa memória comum, para que a CPU possa buscá-los durante a execução.

Tudo vira 0 e 1: o sistema binário

Circuitos eletrônicos só reconhecem dois estados — por isso os computadores representam tudo em binário.

Sistema decimal

0 1 2 3 4 5 6 7 8 9

10 símbolos diferentes

Sistema binário

0 1

Apenas 2 símbolos: ligado / desligado

Convertendo binário → decimal

1 0 1 1

1×8

0×4

1×2

1×1

$$8 + 0 + 2 + 1 = 11$$

$$1011_2 = 11_{10}$$

Bit = menor unidade de informação (0 ou 1) · Byte = 8 bits

Hexadecimal: binário de forma compacta

Sequências binárias longas são pouco práticas para humanos mas o hexadecimal ajuda com isso.

16 símbolos

0 1 2 3 4 5 6 7 8 9 A B C D E F

A=10 B=11 C=12 D=13 E=14 F=15

Cada dígito hexadecimal representa exatamente 4 bits.

Binário: 1010 1111

Hexadecimal: A F

$1010\ 1111_2 = AF_{16}$

Onde aparece no dia a dia:

- Endereços de memória
- Depuração de programas (debugging)
- Programação de baixo nível
- Representação de bytes
- Identificadores técnicos
- Cores em design (#AF00FF, por ex.)

Dos circuitos ao código que escrevemos

Existe uma distância entre o que o hardware executa e o que um programador escreve. Linguagens de programação existem para reduzir essa distância.

01

Instruções de máquina

Sequências de bits ligados e desligados que a CPU compreende.

```
010010 011110...
```



02

Linguagem de baixo nível

A linguagem Assembly surgiu no final da década de 1940, os comando são representados como... mnemônicos!

```
mov eax, 10  
add eax, 20
```



03

Linguagens de médio nível

C, C++ e Rust: São linguagens que se aproximam do alto nível mas nos dão “poder” de baixo nível para controle do hardware.

```
int soma = a + b;  
cout << soma << “\n”;
```



04

Linguagens de alto nível

Python, JavaScript e Lua por exemplo, são linguagens muito mais próximas da linguagem humana

```
media = (n1+n2+n3)/3  
print(media)
```

Quanto mais alto o nível, menos precisamos pensar nos detalhes do hardware — e mais perto ficamos da lógica que queremos expressar.

Python: alto nível, foco na lógica

O código Python não é executado “ao pé da letra” pela CPU — ele passa por um interpretador que traduz cada instrução.

```
print("Olá, mundo!")
```

Por baixo dessa linha simples existem várias camadas:

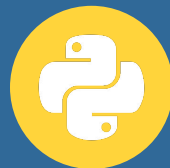
Python

Runtime / Bibliotecas

Sistema Operacional

Instruções de máquina

CPU e memória



Vantagens para quem está começando

- Sintaxe próxima da linguagem humana
- Menos detalhes técnicos para se preocupar de início
- Ideal para focar em lógica, não na verbosidade
- Muito usado no mercado: web, dados, automação, Cripto

O que levamos desta aula



Programação

É construir instruções claras para resolver problemas.



Pensamento computacional

Decomposição, padrões, abstração e algoritmos.



Hardware e Software

A parte física e o conjunto de instruções que a comandam.



Entrada → Processamento → Saída

O ciclo básico de qualquer sistema computacional.



Binário e hexadecimal

Como a informação é representada dentro da máquina.



Níveis de abstração

Da máquina ao Python — cada nível existe para nos aproximar da lógica humana.

Próxima aula: Algoritmos e lógica

Agora que entendemos como o computador “pensa”, o próximo passo é aprender a transformar um problema em uma sequência de passos que ele consiga executar.



Na próxima aula

- O que é um algoritmo, na prática
- Pseudocódigo e fluxograma (introdutório)
- Primeiros passos para pensar “como o Python pensa”



Sobre o curso

- Encontros às segundas e quartas, 11h10–12h30
- 14 encontros presenciais + 2 atividades a distância
- Frequência mínima de 75% para certificação
- Certificado emitido pela COPER/FMP



Perguntas?

Obrigado pela atenção — vamos construir juntos a lógica de programação, um passo de cada vez.